

Introduction to Reinforcement Learning

Daniel Wehner, M.Sc.

SAP SE / DHBW CAS Heilbronn

Summer term 2026



Find all slides on [GitHub](https://github.com/DaWe1992/Applied_ML_Fundamentals) (DaWe1992/Applied_ML_Fundamentals)

Lecture Overview

- | | | | |
|-------------|-----------------------------------|--------------|------------------------------|
| I | Machine Learning Introduction | IX | Evaluation |
| II | Optimization Techniques | X | Decision Trees |
| III | Bayesian Decision Theory | XI | Support Vector Machines |
| IV | Non-parametric Density Estimation | XII | Clustering |
| V | Probabilistic Graphical Models | XIII | Principal Component Analysis |
| VI | Linear Regression | • XIV | Reinforcement Learning |
| VII | Logistic Regression | XV | Advanced Regression |
| VIII | Deep Learning | | |

Agenda for this Unit

① Introduction to Reinforcement Learning

② Dynamic Programming Techniques

③ Miscellaneous

④ Advanced Algorithms

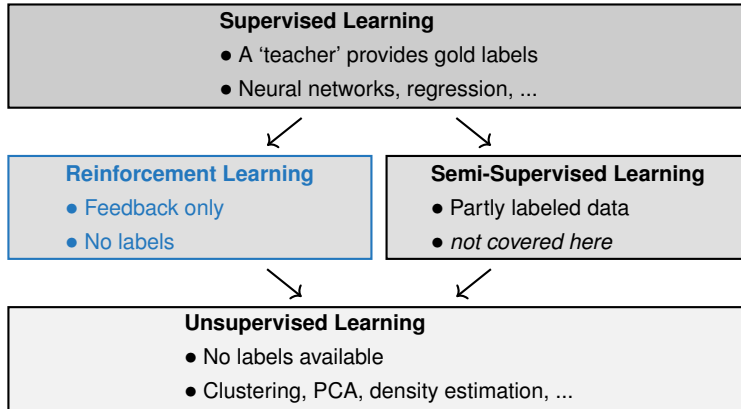
⑤ Wrap-Up

Section:

Introduction to Reinforcement Learning

What is Reinforcement Learning?
Key Challenges in Reinforcement Learning
MENACE (Matchbox Educable Noughts and Crosses Engine)
Formalization of Reinforcement Learning

Dimensions of Learning: Type of Training Information



What is Reinforcement Learning?

- Agent versus (unknown) environment
- The goal is to learn **optimal decisions based on feedback** provided by the environment
- The environment **rewards** the agent for its actions, but **does not** reveal the correct solution (*rewards can be positive or negative*)
- **Applications**
 - Games
 - Tic-Tac-Toe: MENACE [MICHIE.1963]
 - Backgammon: TD-Gammon [TESAURO.1995]
 - Robot control (*robot_{en}* derived from *robota_{cz}* which translates to 'labor service')



Reward Hypothesis

- **Fundamental assumption:** All goals of an agent can be explained by a **single scalar, called the reward**
- It is the RL practitioners' task to find the right set of rewards from which the agent learns the desired behavior
- This is referred to as **reward shaping**
(comparable to feature engineering in data science)

Reward hypothesis: What we mean by goals and purposes can be well thought of as the **maximization of the expected value of the cumulative sum of a received scalar signal** *(called reward)*.



Digression into Psychology: Operant Conditioning (B. F. SKINNER)

Reinforcement learning is comparable to operant conditioning (*e. g. when training a dog*):

	Punishment	Reinforcement
Positive	add noxious stimulus to decrease behavior	add pleasant stimulus to increase behavior
Negative	remove pleasant stimulus to decrease behavior	remove noxious stimulus to increase behavior



Key Challenges in Reinforcement Learning

1 Delayed rewards

- Usually, there is a sequence of actions, before a reward is given
- Problem of **temporal credit assignment**: Which actions were good / bad?

2 Exploitation versus exploration

- Should the agent perform actions which are known to be good...
- ...or should it try out new (*possibly even better*) actions?

3 Partially observable states (*not everything may be observable*)

4 Life-long learning

- The environment may not be static, it could change!
- The agent needs to adapt to these changes...



Credit Assignment Problem

Fundamental problem in reinforcement learning:

- Especially in games: The reward is given at the end of the match
(have I won or lost?)
- **Central Question:** Which moves contributed to winning or losing?
- This problem is referred to as the **credit assignment problem**

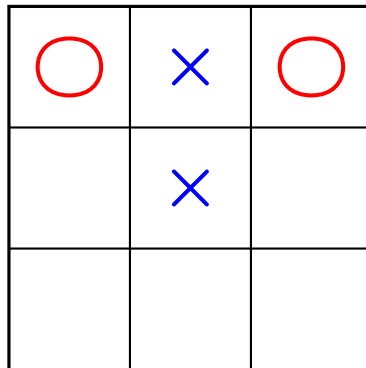
Simple solution: All moves of the sequence are rewarded or punished equally. After many matches, the algorithm converges *(bad moves will be reinforced less positively, good moves will be positively reinforced more often)*.



MENACE: Learning to play 'Tic-Tac-Toe'

- Introduced by [MITCHIE.1963]
- The agent learns to play 'Tic-Tac-Toe'
- MENACE is short for:

Matchbox
Educable
Noughts
And
Crosses
Engine





MENACE: Setup

‘Hardware’:

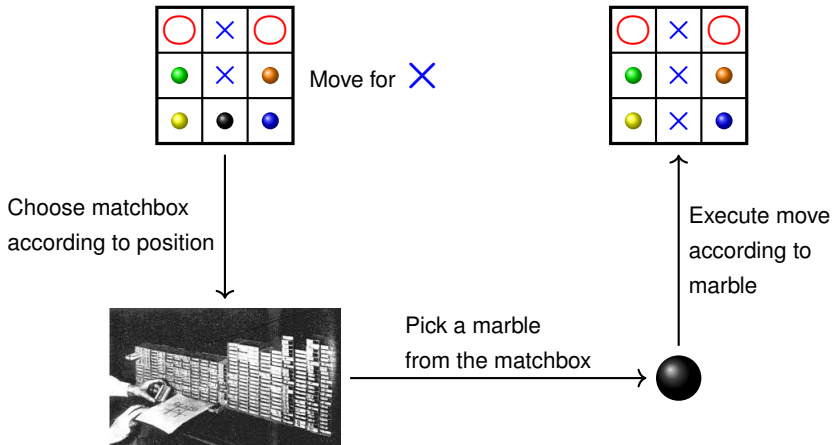
- 287 matchboxes (*one per playing position*)
- Marbles in nine different colors (*one color per field*)
- Initially, each matchbox contains the same amount of marbles of each color

How to perform a move?

- Choose matchbox corresponding to current playing position
- Pick a marble randomly from the matchbox
- Put marker (X or O) on the field that corresponds to its color



MENACE: Visualized





Reinforcement Learning in MENACE

- **Initialization:** Each matchbox contains the same amount of marbles of each color (*all moves are equally likely*)
- **Learning algorithm:**
 - Game **lost**: Marble is removed (*negative reinforcement*)
 - Game **won**: Marble of corresponding color is added (*positive reinforcement*)
 - **Remis**: Marbles are put back into the matchbox (*no change*)

Result:

Probability of successful moves is **increased** ↑

Probability of bad moves is **decreased** ↓



State and Action Spaces, Transition and Reward Functions

Definitions and Notation:

$s \in \mathcal{S}$	State space (<i>discrete or continuous</i>); set of possible states the agent can be in
$s_0 \in \mathcal{S}_0 \subseteq \mathcal{S}$	Set of initial states (<i>'where the agent can be spawned'</i>)
$a \in \mathcal{A}$	Action space (<i>discrete or continuous</i>); set of possible actions
$\delta : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$	State transition function (<i>deterministic or stochastic</i>)
$r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$	Reward function

MARKOV property / MARKOV decision process (MDP): Rewards and state transitions depend on the previous state only, and not how you got into this state (*earlier states and actions do not matter*).



The Policy ω

- Based on the current state s , the agent chooses an action a according to a **policy** ω (*varpi*):

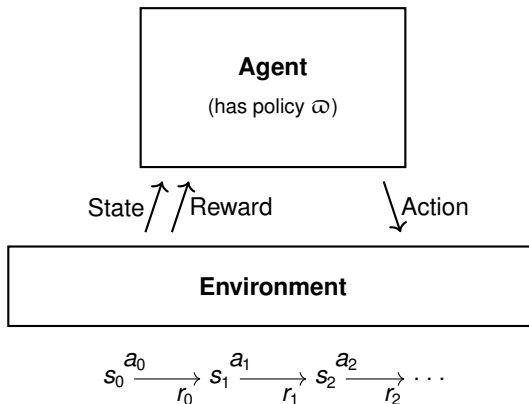
$$\omega : \mathcal{A} \rightarrow \mathcal{A}, \quad a = \omega(s) \tag{1}$$

- The agent reaches a new state s' via the state transition function δ :

$$s' = \delta(s, \omega(s)) \tag{2}$$

- The policy ω is learned by **maximizing the cumulative reward**
- The agent receives a reward in some states (*not necessarily in all states; see temporal credit assignment problem*)

Reinforcement Learning: Formalization (Ctd.)





Learning Task

Goal: Find a policy ϖ^* which maximizes the sum of future rewards

- Maximize the **cumulative reward** R for the **trajectory** τ^{ϖ} a policy ϖ generates
 - **Stochastic environment:** τ can differ (*random elements included*)
 - **Deterministic environment:** τ is always the same (*for same initial state*)

$$R(\varpi) := R(\tau^{\varpi}) := \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \quad (3)$$

- The **discount factor** γ gives more weight to immediate rewards, and exponentially less weight to rewards further in the future

Discount Factor γ

- According to equation (3), rewards t time steps in the future are discounted exponentially by a factor of γ^t
- This means we **prefer immediate rewards** over rewards in the distant future
- **Choice of γ :**
 - Possible range: $0 \leq \gamma \leq 1$
 - $\gamma = 0$: Only immediate rewards are taken into account
(we define $0^0 := 1$ in this case)
 - $\gamma = 1$: Future and immediate rewards are given the same weight
 - Usually, γ is set to a value close to 1, e.g.: 0.99, 0.98, 0.95, ...

Value Function: Value V of a State

$$\begin{aligned} R(\tau^{\varpi}) &= \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \\ &= r(s_0, a_0) + \gamma r(s_1, a_1) + \gamma^2 r(s_2, a_2) + \dots \\ &= r(s_0, \varpi(s_0)) + \sum_{t=1}^{\infty} \gamma^t r(\delta(s_{t-1}, \varpi(s_{t-1})), \varpi(s_t)) \\ &=: V^{\varpi}(s_0) \end{aligned} \tag{4}$$

$V^{\varpi}(s_0)$ is called the ‘value’ of the first state s_0 . It represents the cumulative reward obtained when starting in state s_0 and then following policy ϖ

Optimal Policies and Value Functions

- The optimal policy is denoted by ω^*
- The optimal policy has the **highest expected value for all states**:

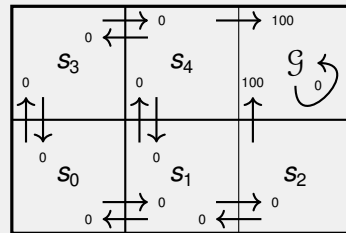
$$\begin{aligned}\omega^*(s) &:= \arg \max_{\omega} V^{\omega}(s) && \forall s \in \mathcal{S} \\ &= \arg \max_{a \in \mathcal{A}} r(s, a) + \gamma V^{\omega^*}(\delta(s, a)) && \forall s \in \mathcal{S}\end{aligned}\tag{5}$$

- Always select the action that maximizes the value function for the next step, when following ω^* afterwards

Problem: Recursive definition of ω^* requires **perfect knowledge**

An illustrative Example [given perfect Knowledge]

- Each square represents a state
- Each arrow represents a possible action
- Small numbers refer to the reward $r(s, a)$ obtained by taking the action
- In this case: Reward is always set to 0, except when entering the goal state $\mathcal{G}$
- $\mathcal{G}$ is an absorbing state (*i. e. it is impossible to get out again*)
- $\gamma := 0.9$



An illustrative Example [given perfect Knowledge] (Ctd.)

In this case we can derive the optimal policy ω^* directly, e. g. $s_0 \rightarrow s_3 \rightarrow s_4 \rightarrow \mathcal{G}$

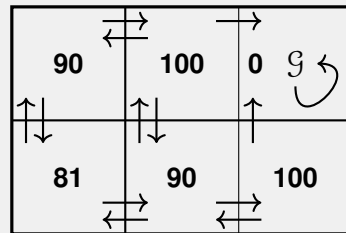
$$V^{\omega^*}(s_0) = 0 + \gamma 0 + \gamma^2 100 + \gamma^3 0 + \dots = \mathbf{81}$$

$$V^{\omega^*}(s_1) = 0 + \gamma 100 + \gamma^2 0 + \gamma^3 0 + \dots = \mathbf{90}$$

$$V^{\omega^*}(s_2) = 100 + \gamma 0 + \gamma^2 0 + \gamma^3 0 + \dots = \mathbf{100}$$

$$V^{\omega^*}(s_3) = 0 + \gamma 100 + \gamma^2 0 + \gamma^3 0 + \dots = \mathbf{90}$$

$$V^{\omega^*}(s_4) = 100 + \gamma 0 + \gamma^2 0 + \gamma^3 0 + \dots = \mathbf{100}$$



Section:

Dynamic Programming Techniques

How to learn the optimal Policy?

Value Iteration

Policy Iteration

Q-Learning

SARSA: State-Action-Reward-State-Action

How to learn the optimal Policy?

- **Without perfect knowledge** it is difficult to learn the optimal policy directly, since training data of the form (s, a) is **not available** ($\neq$ supervised learning)
- The only information at the agent's disposal is the **sequence of observed rewards**:

$$r_0 \longrightarrow r_1 \longrightarrow r_2 \longrightarrow r_3 \longrightarrow r_4 \longrightarrow \dots$$

- **What evaluation function should be learned?**
 - One obvious choice is V^{ω^*}
 - The agent should prefer state s_1 over s_2 , if $V^{\omega^*}(s_1) > V^{\omega^*}(s_2)$

How to learn the optimal Policy? (Ctd.)

Problem:

- In order to calculate V^{ω^*} , the agent needs to know $\delta(s, a)$ and $r(s, a)$
- Basically, ω^* must be known in advance (*deadlock*)

Possible solutions: Use **dynamic programming** techniques like

- Value iteration, policy iteration
- Q-Learning, SARSA



Value Iteration Algorithm

Input: Environment $\mathcal{E} = (\mathcal{S}, \mathcal{A})$, threshold ε

- 1 Initialize $V(s) \in \mathbb{R}$ arbitrarily for all $s \in \mathcal{S}$
- 2 **while** $\Delta > \varepsilon$ **do**
- 3 $\Delta \leftarrow 0$
- 4 **foreach** $s \in \mathcal{S}$ **do**
- 5 $v \leftarrow V(s)$ and $V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$
- 6 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
- 7 **end**
- 8 **end**
- 9 **return** ϖ^* such that $\varpi^*(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')] \quad \forall s \in \mathcal{S}$

Policy Iteration

- **Basic idea:** Initialize a policy randomly, find its value function, update and iterate
 - ① **Policy evaluation:** Find the value function V^{ω} of the current policy ω
 - ② **Policy improvement:** Find a better policy ω' based on the current one
- Select the action that **maximizes the value function** of the current policy ω :

$$\omega'(s) = \arg \max_{a \in \mathcal{A}} r(s, a) + \gamma V^{\omega}(\delta(s, a)) \quad (6)$$

- This leads to a sequence of **improving policies**:

$$\omega^0(s) \longrightarrow V^{\omega^0}(s) \longrightarrow \omega^1(s) \longrightarrow V^{\omega^1}(s) \longrightarrow \dots \longrightarrow \omega^*(s)$$



Policy Improvement Theorem

Very important:

Policy improvement theorem:

If it is true that selecting the first action in each state according to a policy ω' and continuing with policy ω is better than always following ω , then ω' is a better policy than ω .



Policy Iteration Algorithm (deterministic Case)

Input: Environment $\mathcal{E} = (\mathcal{S}, \mathcal{A})$

```
1 Initialize  $V(s) \in \mathbb{R}$  and  $\varpi(s) \in \mathcal{A}$  arbitrarily for all  $s \in \mathcal{S}$ 
2  $policy\_stable \leftarrow false$ 
3 while  $\neg policy\_stable$  do
4    $V_{new} \leftarrow PolicyEvaluation(\varpi, V, \varepsilon = 0.001)$ 
5    $\varpi, policy\_stable \leftarrow PolicyImprovement(V_{new})$ 
6    $V \leftarrow V_{new}$ 
7 end
8 return  $\varpi^*$ 
```



Sub-Algorithm: PolicyEvaluation

Input: Current policy ω , current value function V , threshold ε

```
1 while  $\Delta > \varepsilon$  do
2    $\Delta \leftarrow 0$ 
3   foreach  $s \in \mathcal{S}$  do
4      $v \leftarrow V(s)$ 
5      $V(s) \leftarrow \sum_{s',r} p(s', r | s, \omega(s)) [r + \gamma V^\omega(s')]$ 
6      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
7   end
8 end
9 return  $V$ 
```

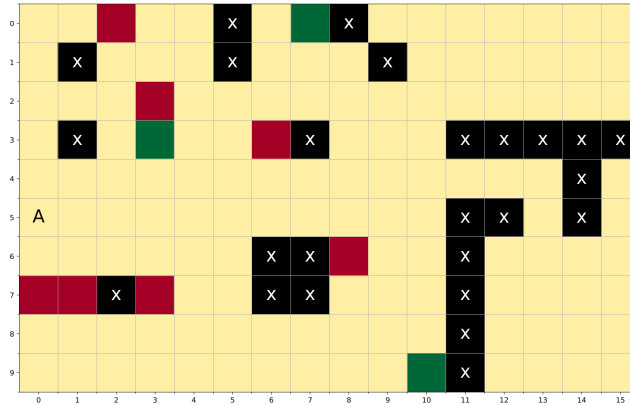


Sub-Algorithm: PolicyImprovement

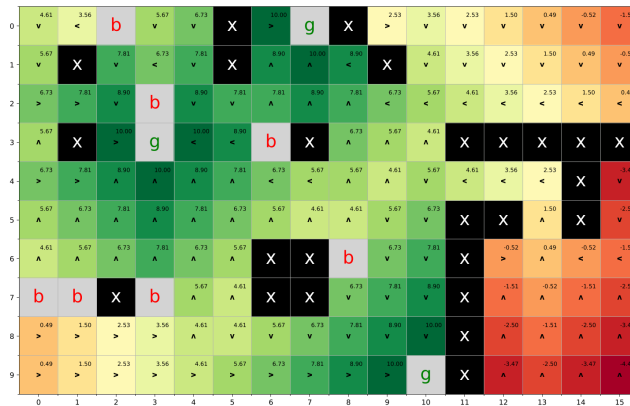
Input: Updated value function V

```
1 policy_stable  $\leftarrow$  true
2 foreach  $s \in \mathcal{S}$  do
3   | old_action  $\leftarrow$   $\omega(s)$ 
4   |  $\omega(s) \leftarrow \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V^\omega(s')]$ 
5   | if old_action  $\neq \omega(s)$  then
6   |   | policy_stable  $\leftarrow$  false
7   | end
8 end
9 return  $\omega, policy\_stable$ 
```

Policy Iteration Example: Grid World



Policy Iteration Example: Grid World (solved)



Q-Learning

- We introduce a function $Q(s, a)$ which represents the **maximum discounted cumulative reward** that can be achieved starting from state s and performing action a as the first action:

$$Q(s, a) := r(s, a) + \gamma V^{\varpi^*}(\delta(s, a)) \quad (7)$$

- Formula (5) for ϖ^* can be rewritten in terms of $Q(s, a)$:

$$\varpi^*(s) = \arg \max_{a \in \mathcal{A}} Q(s, a) \quad (8)$$

- Learning the Q -function, the agent can **select optimal actions without any knowledge about r and δ**

Q-Learning (Ctd.)

- We maximize the expected cumulative reward by always choosing the action a^* with the maximum Q-value:

$$a^* := \arg \max_{a \in \mathcal{A}} Q(s, a)$$

- What may seem surprising: **Local optimal choices lead to a global optimal solution**

Learning the Q-function corresponds to learning the optimal policy ϖ^*



Algorithm for Q-Learning

- We learn the Q -function by **iterative approximation**
- Note the close relationship of Q and V^{ω^*} :

$$V^{\omega^*}(s) = \max_{a' \in \mathcal{A}} Q(s, a') \quad (9)$$

- Therefore, $Q(s, a)$ introduced in equation (7) can be expressed in terms of itself:

BELLMAN equation:

$$Q(s, a) = r(s, a) + \gamma \max_{a' \in \mathcal{A}} Q(\delta(s, a), a') \quad (10)$$

Algorithm for Q-Learning (Ctd.)

- $\hat{Q}$ denotes the estimate of the true Q -value
- The Q -function is represented by a table in the case of **discrete states and actions**
- Initially, the table is filled with random numbers or zeros

State-action pair	$\hat{Q}$ -value
(s_0, a_0)	$\hat{Q}(s_0, a_0)$
(s_0, a_1)	$\hat{Q}(s_0, a_1)$
(s_1, a_0)	$\hat{Q}(s_1, a_0)$
(s_1, a_1)	$\hat{Q}(s_1, a_1)$
...	...

Note: The table may become large if there are many state-action pairs. We need other methods **for continuous states and actions** (*e. g. neural networks*)

Algorithm for Q-Learning (Ctd.)

- The agent repeatedly observes the current state s , chooses some action a , executes it and observes r as well as the new state s'
- Update the table entry for $\hat{Q}(s, a)$ according to the BELLMAN equation (10)
- The agent doesn't have to know $\delta(s, a)$ and $r(s, a)$
- Instead, it executes some actions and observes what happens (**trial-and-error**)
- In the limit, $\hat{Q}$ will converge towards the actual Q -function **if...**
 - ...the system can be modeled as a deterministic MDP...
 - ...**and** the reward function is bounded...
 - ...**and** each state-action pair is visited infinitely often

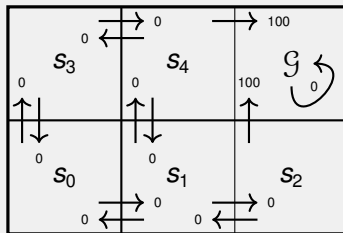


Algorithm for Q-Learning (Ctd.)

```
1 foreach ( $s, a$ ) do
2   |  $\widehat{Q}(s, a) \leftarrow 0$ 
3 end
4 while not converged do
5   | Observe current state  $s$ 
6   | Select an action  $a$  and execute it
7   | Receive the immediate reward  $r$ 
8   | Observe the new state  $s'$ 
9   | Update the table entry for  $\widehat{Q}(s, a)$ :  $\widehat{Q}(s, a) \leftarrow r + \gamma \max_{a' \in \mathcal{A}} \widehat{Q}(s', a')$ 
10  |  $s \leftarrow s'$ 
11 end
```

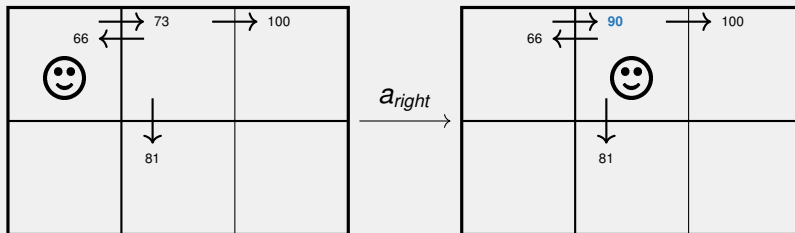
Q-Learning: An illustrative Example

Let us again consider the basic grid world example:



We will now initialize the agent denoted by ☺ in state s_3

Q-Learning: An illustrative Example (Ctd.)



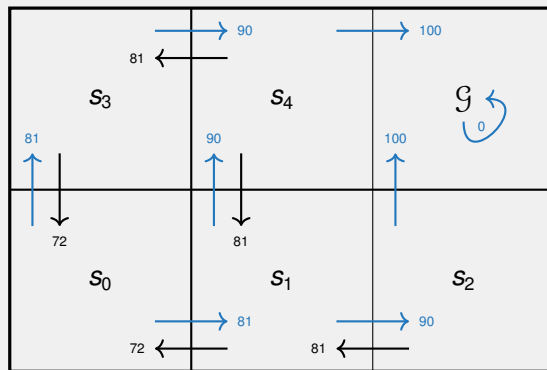
$$\begin{aligned}
 \hat{Q}(s_3, a_{right}) &\leftarrow r + \gamma \max_{a' \in \mathcal{A}} \hat{Q}(s_4, a') \\
 &\leftarrow 0 + 0.9 \max_{a' \in \mathcal{A}} \{66, 81, 100\} \\
 &\leftarrow 90
 \end{aligned}$$

Q-Learning: An illustrative Example (Ctd.)

- Each time the agent makes a move, the Q -value estimates are **propagated backwards** from the new state s' to the old state s
- The immediate reward is used to **augment** the propagated values of $\hat{Q}$
- Since there is an absorbing goal state $\mathcal{G}$, there will be a series of episodes
- **One episode:**
 - ➊ Initialize the agent at a random state
 - ➋ The episode ends as soon as the goal state $\mathcal{G}$ has been reached
 - ➌ Repeat until convergence (go back to ➊)

Q-Learning: An illustrative Example (Ctd.)

Final Q-values:



SARSA: State-Action-Reward-State-Action

- The SARSA algorithm is very similar to Q-learning
- SARSA is short for '**S**tate-**A**ction-**R**eward-**S**tate-**A**ction'
- It refrains from calculating the 'max' in the BELLMAN equation (10) and instead uses the current policy ϖ (*on-policy update*)

SARSA update rule:

$$\hat{Q}(s, a) \leftarrow r(s, a) + \gamma \hat{Q}(\underbrace{\delta(s, a)}_{=s'}, \varpi(s')) \quad (11)$$

Section: Miscellaneous

Exploitation vs. Exploration
Non-Deterministic Rewards and Actions
Temporal Difference Learning
Demo Applications



Exploitation vs. Exploration

When always picking the best action, the **agent overcommits to actions** found to be good during early training, but it fails to explore other actions that might be even better

Exploitation:

- Use the action the agent assumes to be the best one
- Approximate the optimal policy

VS.

Exploration:

- 'Optimal action' may be wrong due to approximation errors
- Try a sub-optimal one

There is a trade-off between exploitation and exploration!

Exploitation vs. Exploration (Ctd.)

Solution: Give all actions a certain probability to be chosen. Two possible options:

- 1 **ϵ -greedy** (small ϵ favours exploitation)

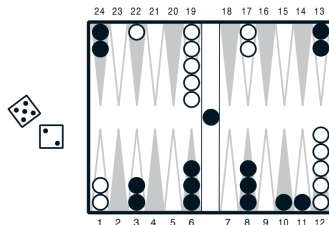
$$p(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & \text{if } a = \arg \max_{a' \in \mathcal{A}} \hat{Q}(s, a') \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise} \end{cases} \quad (12)$$

- 2 **Softmax** (actions with high $\hat{Q}$ -values get a higher probability)

$$p(a|s) = \frac{k^{\hat{Q}(s,a)}}{\sum_j k^{\hat{Q}(s,a_j)}} \quad k > 0 \quad (\text{large } k \text{ favours exploitation}) \quad (13)$$

Non-Deterministic Rewards and Actions

- So far we have only considered the deterministic case
- **What if the environment is non-deterministic, e. g. stochastic?**
 - E. g. in Backgammon, each move involves a roll of the dice $\Rightarrow \delta(s, a)$ varies
 - Also, the rewards may change $\Rightarrow r(s, a)$ varies



Non-Deterministic Rewards and Actions (Ctd.)

- $\delta(s, a)$ and $r(s, a)$ can be viewed as first producing a **probability distribution over the outcomes** based on s and a , and then **drawing an outcome at random** according to this distribution
- If these probability distributions depend solely on s and a (*and not on earlier states or actions*), then the system is called **non-deterministic MARKOV decision process**

The Q -learning algorithm has to be modified in order to be able to handle the non-deterministic case

Non-Deterministic Rewards and Actions (Ctd.)

- Set V^{ω} to be the expected value (solve e. g. by using **Monte Carlo sampling**):

$$V^{\omega}(s_0) := \mathbb{E} \left\{ \sum_{t=0}^{\infty} \gamma^t r(s_t, \omega(s_t)) \right\} = r(s_0, \omega(s_0)) + \underbrace{\frac{1}{k} \sum_{i=1}^k \sum_{t=1}^{\infty} \gamma^t r(s_{t,i}, \omega(s_{t,i}))}_{k \text{ samples}} \quad (14)$$

- The definition of Q has to be updated as well:

$$\begin{aligned} Q(s, a) &:= \mathbb{E} \{ r(s, a) + \gamma V^{\omega^*}(\delta(s, a)) \} \\ &= \mathbb{E} \{ r(s, a) \} + \gamma \mathbb{E} \{ V^{\omega^*}(\delta(s, a)) \} \\ &= \mathbb{E} \{ r(s, a) \} + \gamma \sum_{s'} p(s'|s, a) V^{\omega^*}(s') \end{aligned} \quad (15)$$

Non-Deterministic Rewards and Actions (Ctd.)

- Again, we express the Q -function recursively:

$$Q(s, a) = \mathbb{E}\{r(s, a)\} + \gamma \sum_{s'} p(s'|s, a) \max_{a'} Q(s', a') \quad (16)$$

- We need a new training rule for the algorithm (*the old one fails to converge **due to changing rewards...***)

$$\hat{Q}_n(s, a) \leftarrow (1 - \alpha_n) \hat{Q}_{n-1}(s, a) + \alpha_n [r + \gamma \max_{a'} \hat{Q}_{n-1}(s', a')] \quad (17)$$

- Where $\alpha_n := \frac{1}{1 + \text{visits}_n(s, a)}$ and $\text{visits}_n(s, a)$ is the number of times the state-action pair was visited

Non-Deterministic Rewards and Actions (Ctd.)

- Updates of $\hat{Q}$ are made **more gradually** than in the deterministic case
- For $\alpha_n = 1$ we get the old training rule
- α_n decreases over time which causes the updates become smaller in later iterations
- Due to the average, the Q -learning algorithm ultimately converges
- Q -learning often requires many thousands of training iterations to converge

E. g. TD-Gammon trained for **1.5 million (!)** Backgammon games.

Temporal Difference Learning

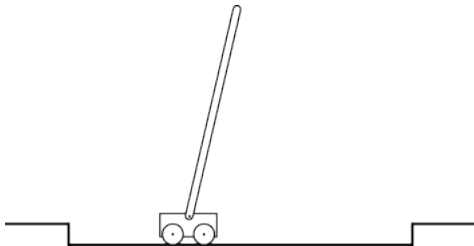
- Q-learning works by iteratively reducing the discrepancy between Q-value estimates **at different time steps** (*based on a one-step lookahead*)
- Q-learning is a special case of **temporal difference learning**:

$$\hat{Q}_n(s, a) \leftarrow (1 - \alpha_n) \hat{Q}_{n-1}(s, a) + \alpha_n [r(s, a) + \gamma \max_{a'} \hat{Q}_{n-1}(s', a')]$$

↕ rewrite the equation

$$\hat{Q}_n(s, a) \leftarrow \hat{Q}_{n-1}(s, a) + \alpha_n \underbrace{\left[\overbrace{r(s, a) + \gamma \max_{a'} \hat{Q}_{n-1}(s', a')}^{\text{after}} - \underbrace{\hat{Q}_{n-1}(s, a)}^{\text{before}} \right]}_{\text{Temporal difference}} \quad (18)$$

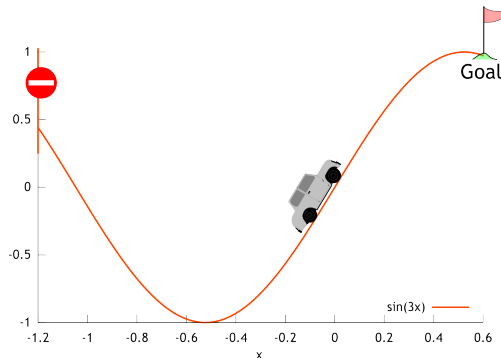
Demo: Cartpole Task



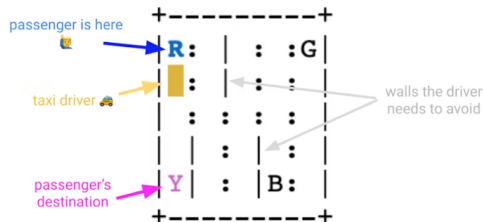
- Move the cart **without** the pole falling over
- Action space $\mathcal{A}$:
 - Move LEFT (0)
 - Move RIGHT (1)
- 4D state space $\mathcal{S}$:
 - Cart position, velocity
 - Angle of pole to cart...
 - ...and its derivative

Demo: Mountain Car Task

- The engine of the car is not strong enough
- You have to go back and forth to acquire momentum
- The car must not hit the wall (left side)
- The car has to reach the top of the hill



Demo: Taxi Task



- Pick up passengers and get them to the drop-off location
- Filled rectangle represents the taxi
- '|' represents a wall which should be avoided
- Blue letter: **Pick-up location**
- Purple letter: **Drop-off location**
- Taxi turns **green** if a passenger is aboard

Section: Advanced Algorithms

Deep Reinforcement Learning

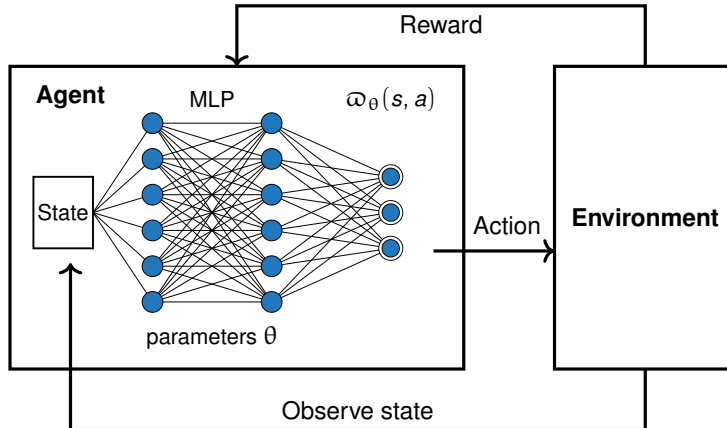
Deep Reinforcement Learning

Keeping track of the Q -function is intractable if state space and/or action space are really large or even continuous. Maybe it does not even fit into memory... We need an alternative!

Idea:

- Use deep learning methods to approximate the policy ω
- E. g. an multi-layer perceptron could predict the action to take given the observed state, the reward, etc.

Deep Reinforcement Learning (Ctd.)



Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih Koray Kavukcuoglu David Silver Alex Graves Ioannis Antonoglou

Daan Wierstra Martin Riedmiller

DeepMind Technologies

{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com

Abstract

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning. The model is a convolutional neural network, trained with a variant of Q-learning, whose input is raw pixels and whose output is a value function estimating future rewards. We apply our method to seven Atari 2600 games from the Arcade Learning Environment, with no adjustment of the architecture or learning algorithm. We find that it outperforms all previous approaches on six of the games and surpasses a human expert on three of them.

<https://arxiv.org/abs/1312.5602>

Section: Wrap-Up

Summary
Recommended Literature
Self-Test Questions
Lecture Outlook

Summary

- **What is reinforcement learning?**
 - Learn optimal decisions based on feedback
 - The agent only knows how good the action was, but not the optimal solution
 - **Credit assignment problem:** Which move(s) is / are responsible for success / failure?
 - Simple example: MENACE
- **Important terms:**
 - State s , action a
 - Policy ϖ (*optimal policy* ϖ^*)
 - Reward r (*immediate reward, discounted reward*)

Summary (Ctd.)

- **Algorithms (dynamic programming)**
 - Value iteration and policy iteration
 - Q-learning
 - SARSA
- **Exploitation versus exploration**
 - **Exploitation:** Exploit what is already known to be good
 - **Exploration:** But don't miss to explore new states and actions
- Non-deterministic case (*the functions $\delta(s, a)$ and $r(s, a)$ are stochastic*)
- Temporal difference learning
- Deep reinforcement learning

Recommended Literature

1 [MITCHELL.1997]

2 [SUTTON.2014]

(For free PDF versions, see list in GitHub readme!)



Self-Test Questions

- 1 Name and explain some challenges in reinforcement learning!
- 2 What is a policy? What is a value function?
- 3 Describe what value iteration is!
- 4 What is policy iteration? What is the difference to value iteration?
- 5 What does the policy improvement theorem state?
- 6 What is Q -Learning and how does it differ from value iteration / policy iteration?
- 7 Briefly explain what the exploitation / exploration trade-off is about!
- 8 Name two strategies which ensure that the agent also chooses sub-optimal actions.
- 9 How do we handle the non-deterministic case?

What's next...?

- | | | | |
|-------------|-----------------------------------|-------------|------------------------------|
| I | Machine Learning Introduction | IX | Evaluation |
| II | Optimization Techniques | X | Decision Trees |
| III | Bayesian Decision Theory | XI | Support Vector Machines |
| IV | Non-parametric Density Estimation | XII | Clustering |
| V | Probabilistic Graphical Models | XIII | Principal Component Analysis |
| VI | Linear Regression | XIV | Reinforcement Learning |
| VII | Logistic Regression | • XV | Advanced Regression |
| VIII | Deep Learning | | |

Thank you very much for the attention!

***** Artificial Intelligence and Machine Learning *****

Topic: Introduction to Reinforcement Learning

Term: Summer term 2026

Contact:

Daniel Wehner, M.Sc.

SAP SE / DHBW CAS Heilbronn

daniel.wehner@sap.com

Do you have any questions?